

Конспект для подготовки к собеседованию

Data Science (на основе видео и ML Handbook Яндекса)

Вот расширенный и глубоко проработанный конспект для подготовки к собеседованию. Я значительно детализировал цитаты из видео, добавил экспертные «фишки», о которых говорит спикер, и обогатил теоретическую часть математическими выкладками и строгими концепциями из ML Handbook. [cite: 1, 2]

1. Как работает логистическая регрессия.

Цитата из видео: «У нас транспонированный вектор весов признаков умножается на наш объект. Мы пропускаем это всё через сигмоиду и получаем вероятность принадлежности классу. <...> Фишечка: расскажите почему именно логлос. То, что это приходит из теории вероятности, из максимизации правдоподобия. Расскажите то, что он выпукл и вы будете всегда сходиться к одному решению. <...> На больших ошибках предсказаний он даёт большой штраф». [cite: 3, 4, 5, 6]

Дополнение из учебника: Логистическая регрессия предсказывает логит (log odds) — логарифм отношения вероятностей:

$$\log \left(\frac{p}{1-p} \right)$$

Чтобы перевести линейный ответ $\langle w, x_i \rangle$ в вероятность, используется сигмоида:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Оптимизация LogLoss — это следствие метода максимального правдоподобия для распределения Бернулли:

$$p(y|X, w) = \prod p_i^{y_i} (1 - p_i)^{1-y_i}$$

Разделяющая поверхность классификатора задается уравнением $\langle w, x \rangle = 0$, что геометрически является гиперплоскостью. [cite: 7]

Резюме: Это линейный классификатор, обучаемый через максимизацию правдоподобия (минимизацию строго выпуклого LogLoss). Модель предсказывает логарифм отношения шансов, который через сигмоиду трансформируется в вероятность, а граница классов образует гиперплоскость. [cite: 8, 9]

Дополнительный пример: В задачах банковского скоринга лог-лос критически важен, так как он сильно штрафует модель за ложную уверенность. Если модель предсказывает вероятность 0.99 для «плохого» заемщика, штраф будет намного выше, чем при предсказании 0.6.

2. Что такое переобучение и как его замечать.

Цитата из видео: «Когда наша модель слишком сильно подстроилась под данные: у нас есть неплохое качество на трейне, но на тесте качество слишком плохое. <...> Более продвинутый ответ — это использовать кросс-валидацию, посмотреть, что модель подстроилась под различные фолды. <...> У каждой модели есть своя специфика борьбы: у деревьев это высота листа, у нейронных сетей — дропаут». [cite: 10, 11, 12]

Дополнение из учебника: В терминах теории ML переобучение описывается через Bias-Variance Decomposition. [cite: 13] Переобученная модель имеет высокий разброс (Variance) — она избыточно чувствительна к случайным флуктуациям и запоминает шум обучающей выборки вместо реального сигнала. [cite: 14]

Резюме: Переобучение (Overfitting) — это состояние модели с высоким Variance, когда она заучивает шум. [cite: 15] Детектируется через разрыв метрик между train и test/-фолдами кросс-валидации. Лечится регуляризацией, зависящей от типа алгоритма. [cite: 16]

3. Фишки: data augmentation, SMOTE, шум.

Цитата из видео: «Круто работать с переобучением через данные: можно добавить шум, можно сделать больше новых данных... использовать SMOTE, чтобы разнообразить ваш датасет. Это всё очень хорошие методы борьбы с переобучением». [cite: 17]

Дополнение из учебника: Аугментация (внесение изменений в данные) — важнейший инструмент регуляризации в глубинном обучении. [cite: 18] Например, поворот картинки самолета на 10 градусов создает новый для сети объект, но с той же меткой. [cite: 19] Это позволяет «дать модели понять, какие преобразования над данными являются допустимыми». [cite: 20] Главное ограничение: аугментированные данные должны принадлежать к той же генеральной совокупности. [cite: 21]

Резюме: Аугментация и искусственное зашумление данных — это мощная форма регуляризации, расширяющая выборку и обучающая модель инвариантности к допустимым в реальном мире искажениям (повороты, шумы). [cite: 22]

4. Почему L1 регуляризация зануляет признаки.

Цитата из видео: «Интуитивное понимание: гиперплоскость L1 регуляризации — это октаэдр, а L2 — сфера. И октаэдр сводится к нулям. <...> Более крутое доказательство — это через субдифференциалы: мы доопределяем производную по модулю. У L2 идут большие штрафы при больших весах, а для модуля градиент более равномерный. Поэтому веса равномерно отнимаются и сходятся к нулю». [cite: 23, 24, 25]

Дополнение из учебника: При L1-регуляризации (Lasso) штраф имеет вид:

$$\lambda \sum |w_j|$$

[cite: 26] Линии уровня L1-нормы — это ромбы (в 2D) или многогранники с острыми вершинами, лежащими строго на осях координат. [cite: 27] Линии уровня основной функции потерь в точке оптимума касаются именно этих углов на осях, обнуляя часть координат. [cite: 28] Оптимизация L1 сложна из-за недифференцируемости модуля в нуле, для чего применяются проксимальные методы градиентного спуска. [cite: 29]

Резюме: Геометрия L1-штрафа (ромбы с острыми углами на осях) и свойства субградиента приводят к тому, что точка касания с функцией потерь попадает на оси координат, создавая разреженные (зануленные) вектора весов, что работает как встроенный feature selection. [cite: 30]

5. Как устроено дерево решений.

Цитата из видео: «Это бинарное дерево. Для категорий есть ответы да/нет. Для вещественных признаков мы разбиваем их на бины, проходим по порядку и смотрим: если признак меньше значения — идет влево, больше — вправо. Мы хотим уменьшать

кроссэнтропию или увеличивать Information Gain (чтобы в листьях была как можно меньше дисперсия)». [cite: 31, 32, 33]

Дополнение из учебника: Дерево задает кусочно-постоянную аппроксимацию. Внутренняя вершина содержит предикат:

$$B_v(x, j, t) = [x_j \leq t]$$

[cite: 34] Жадный алгоритм на каждом шаге максимизирует критерий ветвления:

$$Branch = |X_m|H(X_m) - |X_l|H(X_l) - |X_r|H(X_r)$$

Информативность (impurity) $H(X)$ зависит от задачи: [cite: 35]

- для регрессии (MSE) это дисперсия ответов: $\frac{1}{|X|} \sum (y_i - \bar{y})^2$
- для классификации — критерий Джини: $\sum p_k(1 - p_k)$
- или энтропия Шеннона: $-\sum p_k \log p_k$

Резюме: Дерево строится жадно: пространство признаков итеративно делится гиперплоскостями, параллельными осям. [cite: 36] Сплит выбирается так, чтобы максимизировать падение неопределенности (дисперсии таргета или энтропии классов) в дочерних узлах. [cite: 37]

6. Переобучение в деревьях, прунинг.

Цитата из видео: «Дерево можно ограничить не с помощью кросс-валидации, а при построении: добавить в лосс наказание плюс α умножить на высоту дерева. Расскажите про прунинг. Можно требовать минимальное количество элементов в листе при разбитии». [cite: 38, 39]

Дополнение из учебника: Деревья — чемпионы переобучения, они могут идеально приблизить обучающую выборку (до 1 объекта в листе). [cite: 40] Регуляризация (early stopping/пре-прунинг) останавливает ветвление при достижении лимитов глубины или размера листа. [cite: 41] Прунинг (стрижка) работает иначе: дерево строится жадно до конца, а затем вершины удаляются с проверкой качества на отложенной выборке. [cite: 42]

Резюме: Деревья легко запоминают шум. С этим борются либо останавливая построение (early stopping), либо отсекая лишние ветви уже построенного глубокого дерева на отложенной выборке (прунинг). [cite: 43]

7. Как работает случайный лес (Random Forest).

Цитата из видео: «Деревья строятся параллельно и независимо друг от друга. Используется бутстрап объектов. Важная фишка: признаки выбираются бутстрапом *на каждом разбиении* (в каждом узле), а не для всего дерева в начале. Лес нужен, чтобы уменьшать дисперсию ошибки (variance), поэтому нам нужны как можно более глубокие, переобученные деревья с минимальным смещением». [cite: 44, 45, 46]

Дополнение из учебника: В основе лежит метод случайных подпространств и бэггинг. В каждой вершине отбираются случайные $n < N$ признаков. [cite: 47] Если базовые глубокие деревья не скоррелированы, то дисперсия ансамбля из k деревьев падает в k раз:

$$\mathbb{V}[a(x, X)] = \frac{1}{k} \mathbb{V}_X b(x, X)$$

Смещение (bias) леса при этом остается равным смещению одного дерева. [cite: 48]

Резюме: Случайный лес — параллельный ансамбль глубоких переобученных деревьев (с низким bias). [cite: 49] Рандомизация по объектам (бутстрап) и признакам делает деревья независимыми, что при усреднении радикально снижает общую дисперсию (variance). [cite: 50]

8. Чем отличается бустинг от Random Forest.

Цитата из видео: «Бустинг строится последовательно, каждая модель учится на ошибку предыдущей. Если выкинуть первую модель, бустинг сломается, а случайный лес — нет. Бустинг уменьшает смещение (bias), поэтому мы хотим, чтобы дисперсия каждой базовой модели была как можно меньше — используются неглубокие деревья (от 3 до 8 уровней)». [cite: 51, 52, 53]

Дополнение из учебника: Градиентный бустинг строит композицию аддитивно. Каждое следующее дерево настраивается на минимизацию антиградиента функции потерь композиции всех прошлых шагов. [cite: 54] В отличие от леса, бустинг требует простых базовых алгоритмов для постепенного снижения Bias, но подвержен риску переобучения. [cite: 55]

Резюме: Лес снижает Variance, усредняя параллельные глубокие деревья. Бустинг снижает Bias, строя последовательность мелких деревьев, каждое из которых корректирует остаточную ошибку предыдущих. [cite: 56]

9. Метрики регрессии и классификации.

Цитата из видео: «Регрессия: MAE, MAPE, MSE, RMSE, SMAPE. Важно говорить, что они значат (MSE подстраивается под выбросы, MAPE нужна для понимания порядка). Классификация: Accuracy, Precision, Recall, F1, F-beta, ROC-AUC, PR-кривая». [cite: 57, 58]

Дополнение из учебника: MSE математически приближает матожидание целевой переменной. [cite: 59] MAE приближает медиану и значительно меньше штрафует за грубые выбросы. [cite: 60] Accuracy неинформативна при дисбалансе классов. [cite: 61]

Резюме: Для регрессии метрики выбираются исходя из отношения к выбросам и необходимости относительных оценок. [cite: 62] В классификации при дисбалансе фокус смещается на метрики полноты, точности и ранжирования. [cite: 63]

10. Что такое ROC-AUC и как его интерпретировать.

Цитата из видео: «ROC-AUC показывает: если мы берём случайный объект из класса 1 и случайный объект из класса 0, наша модель их правильно отранжировала. Вопрос-ловушка: как изменится ROC-AUC, если предсказания 0.8 и 0.3 заменить на 0.9 и 0.7? Ответ: никак, нам не важна сама вероятность, важно лишь расположение». [cite: 64, 65, 66]

Дополнение из учебника: Это площадь под кривой (Area Under Curve), отражающей зависимость True Positive Rate от False Positive Rate при всевозможных порогах. [cite: 67]

Резюме: ROC-AUC — это вероятность правильного попарного ранжирования: случайно взятый положительный объект получит скор выше, чем случайно взятый отрицательный. [cite: 68]

11. Precision — как объяснить на собеседовании.

Цитата из видео: «Точность. Нарисуйте для себя Confusion Matrix. Пример: выбор места для постройки дома. Нам не страшно пропустить хорошие места, но очень важно, чтобы то место, которое мы определили как "хорошее гарантированно не было подвержено катаклизмам». [cite: 69, 70, 71]

Дополнение из учебника: Доля истинно положительных среди всех объектов, названных классификатором положительными. [cite: 72] Служит для контроля ошибок первого рода.

Резюме: Точность измеряет "надежность" положительных срабатываний модели. [cite: 73] Метрика критична там, где цена ложной тревоги катастрофически высока. [cite: 74]

12. Recall и его применение в медицине.

Цитата из видео: «Пример: в медицине нам очень важно не пропустить больного. Для нас не так страшно, если мы здорового отправим на обследование, главное — человек выживет». [cite: 75, 76]

Дополнение из учебника: Полнота (Recall) характеризует способность модели находить целевые объекты из всего множества реально существующих положительных примеров. [cite: 77] Фокусируется на минимизации ошибок второго рода.

Резюме: Полнота измеряет "охват" класса. [cite: 78] Ее максимизируют, когда пропуск события обходится намного дороже, чем ложное срабатывание. [cite: 79]

13. Связь Recall и ROC-AUC.

Цитата из видео: «Фишечка: Recall на самом деле используется в ROC-AUC. Осями там выступают True Positive Rate (это Recall для класса 1) и False Positive Rate». [cite: 80, 81]

Дополнение из учебника: Кривая строится в координатах (FPR, TPR) при изменении порога уверенности. TPR математически тождественна метрике Recall. [cite: 82]

Резюме: ROC-кривая буквально состоит из метрик полноты: она показывает, как меняется полнота нахождения целевого класса ценой потери полноты правильного нахождения отрицательного класса. [cite: 83]

14. Зачем нужна PR-кривая и в чём её отличие от ROC-AUC.

Цитата из видео: «При суперсильном дисбалансе классов PR-кривая показывает себя лучше. ROC-AUC более оптимистичен, а PR-кривая делает акцент именно на положительных таргетах». [cite: 84, 85]

Дополнение из учебника: PR-кривая концентрируется исключительно на качестве предсказания меньшего (положительного) класса, игнорируя успехи в нахождении True Negatives. [cite: 86]

Резюме: В условиях жесткого дисбаланса ROC-AUC дает обманчиво высокие оценки. [cite: 87] PR-кривая фокусируется только на миноритарном классе, давая объективную картину.

15. Подбор гиперпараметров (GridSearch, RandomSearch, Optuna).

Цитата из видео: «GridSearch не подходит, если большой набор значений. RandomSearch

берет случайные значения. Байесовская оптимизация строит распределение потенциальной награды». [cite: 88, 89]

Дополнение из учебника: GridSearch страдает от размерности. RandomSearch эффективнее находит оптимум (60 итераций дают вероятность 95% попасть в топ-5% лучших решений). [cite: 90] Байесовская оптимизация учится на истории запусков. [cite: 91]

Резюме: Перебор по сетке слишком долгий. Случайный поиск работает лучше за счет исследования пространства. [cite: 92] Индустриальный стандарт — Байесовская оптимизация (Optuna). [cite: 93]

16. Как оценить важность признаков (feature importance) в модели.

Цитата из видео: «В бизнесе важно показывать, почему ML работает. Линейные: коэффициенты. Деревья: индекс Джини. Permutation Importance: перемешали значения фичи, посмотрели на качество. SHAP values — на основе теории игр». [cite: 94, 95, 96]

Дополнение из учебника: В линейной модели веса имеют смысл только при отсутствии мультиколлинеарности. [cite: 97, 98]

Резюме: Базовые методы — веса или InfoGain. Универсальные методы — Permutation и SHAP. [cite: 99] SHAP математически самый точный, но вычислительно тяжелый.

17. Как работать с категориальными фичами (и чем хорош CatBoost).

Цитата из видео: «Базовые: Label Encoding, One-Hot, Frequency, Target Encoding. CatBoost хорош тем, что сам берет пары категорий и использует Ordered boosting». [cite: 100, 101, 102]

Дополнение из учебника: При классическом One-Hot возникает риск мультиколлинеарности (dummy variable trap). [cite: 103]

Резюме: Простые методы кодируют категории числами или векторами. Продвинутое модели (CatBoost) генерируют комбинации и используют динамическое таргет-кодирование. [cite: 104]

18. В чём разница между == и is в Python.

Цитата из видео: «Базовый ответ: == смотрит на равенство значений, а is проверяет, один это объект или нет (адреса памяти)». [cite: 105]

Дополнение из учебника: Различие между сравнением по значению и по идентификатору — ключевой инженерный нюанс Python.

Резюме: == проверяет данные, а is проверяет идентичность ссылок. [cite: 107]

Пример на Python:

```
x = [1, 2, 3]
y = [1, 2, 3]
print(x == y) # True (
)
print(x is y) # False (
)
```

19. Как устроен dict в Python и что такое коллизии.

Цитата из видео: «Dict — это хэш-мапа. Асимптотика в среднем $O(1)$. Коллизия — когда для разных ключей получается один индекс. В Python используется открытая адресация». [cite: 108, 109, 110]

Резюме: Словарь — хэш-таблица на открытой адресации. [cite: 111, 112] Массив расширяется в 2 раза при заполнении на 66%. [cite: 113]

20. Что такое итераторы и зачем они нужны.

Цитата из видео: «Итератор — объект, позволяющий обойти коллекцию, не держа её в памяти. Обязательны `__iter__` и `__next__`». [cite: 114, 115]

Дополнение из учебника: Итераторы реализуют концепцию "ленивых" вычислений. [cite: 116] Это критично для Big Data. [cite: 117]

Резюме: Итераторы экономят оперативную память, вычисляя элементы по одному "на лету". [cite: 118]

21. Теория вероятностей: задача на сумму 11.

Цитата из видео: «Какая вероятность того, что при подбрасывании двух шестигранных кубиков выпадет сумма 11?». [cite: 119]

Резюме: Общее количество исходов: $6 \times 6 = 36$. [cite: 120] Благоприятных исхода только два: (5, 6) и (6, 5). [cite: 121] Вероятность $P = 2/36 = 1/18$.

22. Теорема Байеса: формула и применение.

Цитата из видео: «Почти всегда выпадает задача на теорему Байеса. Просто заучите формулу». [cite: 122, 123]

Дополнение из учебника: Формула:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Формула корректирует априорные ожидания с учетом новых фактов. [cite: 124]

Резюме: Переоценивает вероятности гипотез при поступлении новых данных. [cite: 125]

23. Что такое p-value.

Цитата из видео: «p-value — это вероятность получить наблюдаемое различие или более экстремальное, если нулевая гипотеза верна». [cite: 126]

Дополнение из учебника: Оценивает противоречие данных нулевой гипотезе в A/B тестах. [cite: 127]

Резюме: Это метрика "удивления". Чем меньше p-value, тем маловероятнее получить результаты случайно. [cite: 128]

24. Статтесты.

Цитата из видео: «t-test для нормального распределения, Манн-Уитни — если есть выбросы, Bootstrap — универсально». [cite: 129, 130]

Резюме: Выбор теста зависит от распределения: параметрические для средних, непараметрические для медиан. [cite: 131, 132]

25. Функции активации и Dropout.

Цитата из видео: «Активации добавляют нелинейность. Dropout борется с переобучением, отключая нейроны на трейне». [cite: 133, 134]

Дополнение из учебника: На инференсе выходы масштабируются на $(1 - p)$. [cite: 135, 136]

Резюме: Активации ломают линейность слоев, а Dropout обучает ансамбль подсетей для повышения устойчивости.